SOFTWARE ARCHITECTURE

Software Development Life Cycle

BEYOND BOUNDARIES

HOA SEN UNIVERSITY
WORLD CLASS EDUCATION

QS STARS RATING SYSTEM

ACBSP ACCREDITED

AUN-QA

MOET ACCREDITED
KIỂM ĐỊNH CHẤT LƯỢNG GIÁO DỤC QUỐC GIA

Content

- What is SDLC?
- The Waterfall model
- The Spiral model
- Agile model
- Microsoft open-source strategy to life cycle
- Summary

What is SDLC?

- SDLC is an industry recognized **process**
- A series of activities or steps
  - To develop a new enterprise application, or
  - To modify the existing software.

Need for a process

- Discipline is an important factor in order to be a successful person in their career.
- They adhere to certain rules and norms at any point of time.
- Software development needs a discipline/process to produce a high-quality enterprise application, which meets or exceeds the expectations of end customers.
- The development effort should be limited to an estimated time and cost.

Insight of core phases

Requirement phase

Design phase

Implementation phase

Testing phase

Production phase

Discussion

- In your opinion, what is the role of each phase in the SDLC?

Insight of core phases

- Requirement phase, the requirements are collected based on the needs of the business stakeholders.
- Design phase describes the structure and components of the new system as a collection of modules and subsystems.
- Implementation is the programming phase in which the design is converted into reality.
- The testing phase is used to certify the functionality of the developed application.
- Production is the final phase to roll out the system for end customer usage.

SDLC models

- The Waterfall model
- The Spiral model
- The Incremental model
- Agile methodology
- …

Discussion

- What other SDLC models do you know?
- What is the difference between “model” and “methodology”?

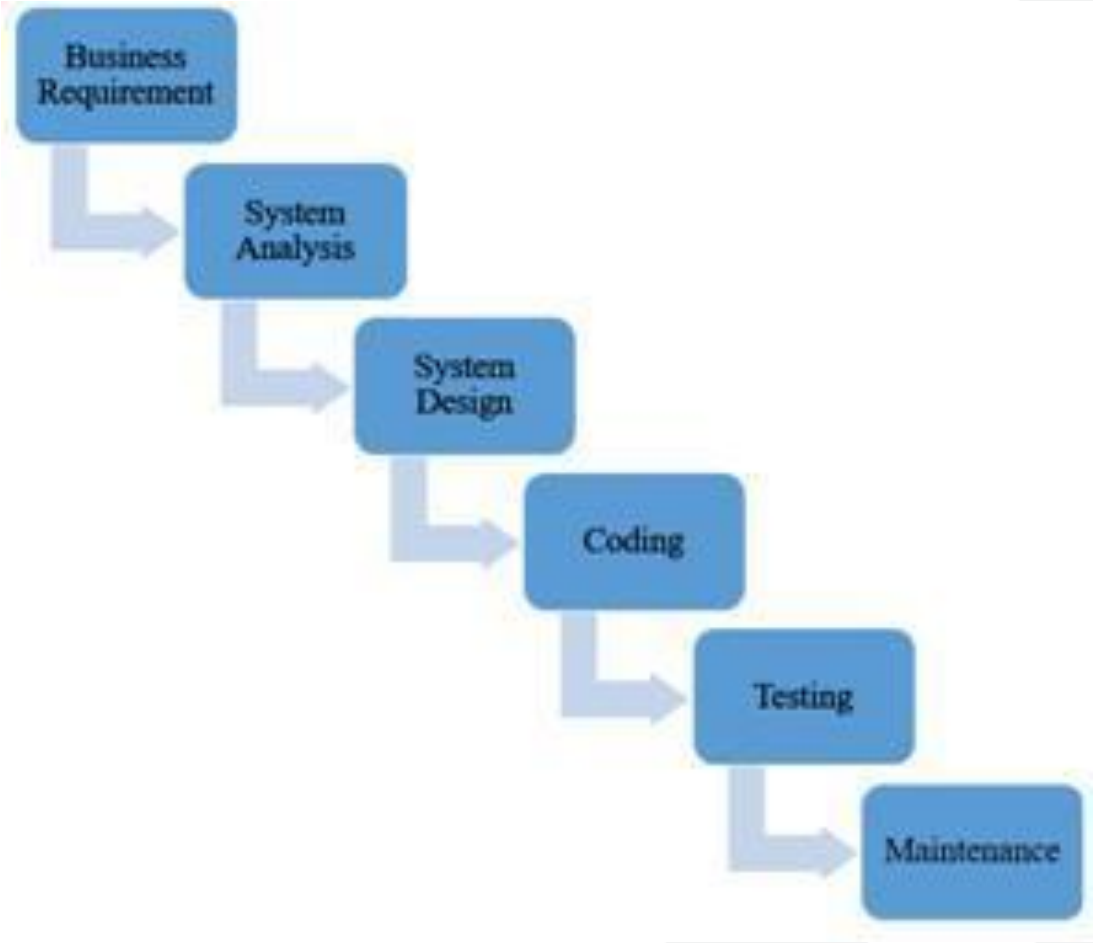
The Waterfall model

- The Waterfall model is developed by dividing the whole process into a few separate phases.
- Each phase of the waterfall mode, is expected to be signed off, before getting into the subsequent phase.
- It involves the deliverable artifact of each phase like design document, business requirement document, source code, test plan, etc.

Core phases

- Business requirement
- System analysis
- System design
- Coding
- Testing
- Maintenance

The Waterfall model



The Waterfall model

- Business requirement
  - All possible specifications and requirements are collected from the business team, and documented as a Business Requirements Document (BRD).
  - It acts as an evidence of the business agreement.
  - Example
    - To build a mobile-based application to check the status of the customer support tickets.
    - The BRD will contain the complete functionalities of the mobile application, supported mobile type, the customer credentials, and so on.

The Waterfall model

- System analysis
  - The BRD specifications are analyzed in detail from the software system point of view.
  - The deliverable document is referred to as a Functional Specification Document (FSD).
  - FSD is more of a technical response against the given BRD
  - Example
    - When the end user clicks the sign-up button, the application routes to the new user registration form.
    - FSD describes the interaction between human and enterprise application, that is, the end user and mobile application.

The Waterfall model

- System Design
  - System Design Document (SDD) is the outcome of the third phase developed by the software design professionals.
  - SDD describes the new system intention or replacement design, with the detailed descriptions of the system architecture and its related components.

The Waterfall model

- System Design (cont.)
  - Two levels of SDD
    - High Level Design (HLD)
    - Low Level Design (LLD)
  - Example
    - SDD is built with the mobile application architecture along with a few internal component designs.
    - The detail design document covers the input format, database design, processing logic, output layouts, operating environment, interface specifications, and so on.

The Waterfall model

- Coding
  - The core phase of implementation is coding.
  - The software developers build the application using a set of technology tools, languages, and the like.
  - The key deliverable of this phase is the source code of the enterprise application
  - Example
    - The actual mobile application development is part of this phase.
    - Objective C is used for iOS devices, and Java is used for Android devices.

The Waterfall model

- Testing
  - Testing is the gateway to validate the functionality, stability, and usability of the developed application.
  - There are multiple types of testing in an enterprise. Integration testing covers the connectivity of the components across the application. System testing covers the complete functionality of the application…

Discussion

- Read the article “The different types of software testing” at https://www.atlassian.com/continuous-delivery/software-testing/types-of-software-testing
- What type of testing is done by the coder/ developer?
- What type of testing is done by the tester?
- What type of testing is done by the user/ customer?

Maintenance

- Once the certified product is rolled into the production environment, the support team takes control of application maintenance.
- They monitor the production version continuously and alert the respective team if anything goes wrong.
- As the monthly update, the support team is supposed to publish the scorecard of the application in the production environment.

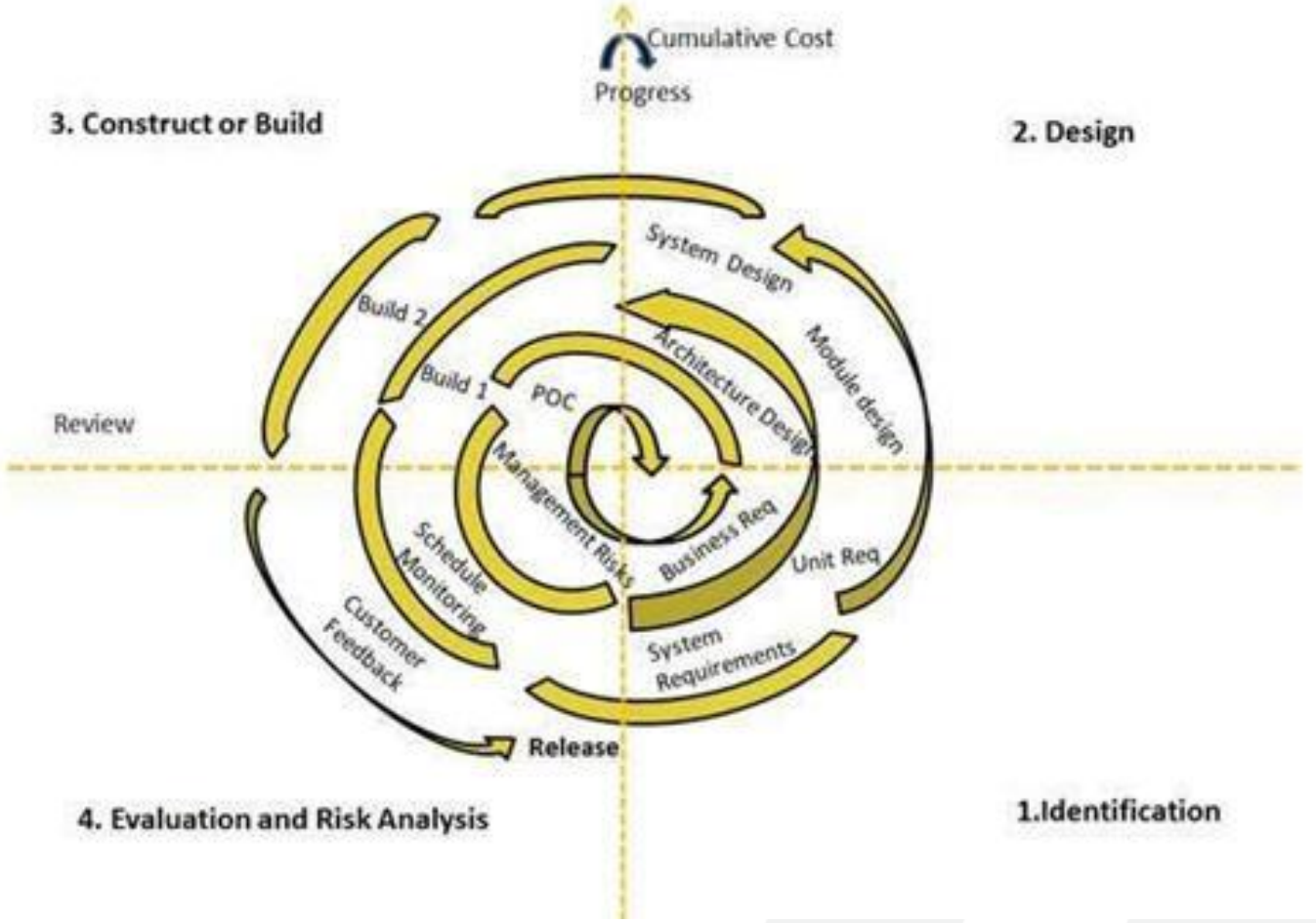
The Spiral model

- Definition
  - The Spiral model is a combination of the **iterative development process** model and **sequential linear development model**, that is, **the waterfall model, with very high emphasis on risk analysis.**

Core phases

- The four phases adopted in the Spiral model are
  - Identification
  - Design
  - Construct or Build
  - Evaluation and Risk Analysis

Core phases


Cumulative Cost
Progress
3. Construct or Build
2. Design
System Design
Build 2
Build 1
POC
Architecture Design
Module design
Review
Management Risks
Business Req
Unit Req
Schedule Monitoring
Customer Feedback
System Requirements
Release
4. Evaluation and Risk Analysis
1.Identification

Identification

- It is essential to find out the purpose of the software along with the measures to be taken.
- It helps to achieve a software to serve all your requirements.

Identification

- The key coverages in the first phase are listed as follows:
  - Studying and gathering requirements
  - Detailed study on the feasibility
  - Reviews run by a walkthrough to streamline the requirements
  - Understanding the requirements document
  - Finalizing the list of requirements
  - Reviewing the project to take the decision to continue with the next spiral
  - If the decision to continue is taken, plans are drafted for the next phase

Design

- This phase calculates all the possible risks with the possibility to encounter; apparently, it is the process to figure out how you would resolve these risks.
- The key coverages in the second phase are listed as follows:
  - The potential risks are identified based on the drafted requirements and brain- storming sessions
  - On identification of the risks, the mitigation strategy is designed and finalized to roll out
  - As an artifact, a document is generated to highlight all the risks and its mitigation plans

Build

- The target software is constructed and tested for its core functionality, reliability, and security aspects.
- As the core engineering phase, the key coverages in this phase are listed as follows:
  - The actual development and testing of the planned software
  - Coding of the product
  - Execution of the test cases and test results
  - Generation of the test summary report and defect report

Evaluation

- After the current cycle has completed to produce the first prototype, the cycle is re-planned and repeated.
- The process proceeds to get into the next prototype 2.
- As the final step of evaluation, these two key steps are executed:
  - Customer evaluates the produced software to provide their feedback for the approval
  - Generating the release document to highlight the implemented features

Comparing the Waterfall and the Spiral models

Sr.No.	Waterfall	Spiral
1	Business requirements are frozen after the initial phase.	Requirements are not frozen by the end of the initial phase. It is kind of executed in a continuous mode.
2	In terms of project execution, there is a high level of risk and uncertainty because of the missing stringent risk management.	By design, the spiral model is modeled to handle better riskmanagement
3	The Waterfall framework type is more of a linear sequential model.	The framework type of the spiral model is based on an iterative process; within each iteration, the linear model is preferred.
4	As the user involvement is only at the beginning of the process, it turns down communication between the customer and developer throughout the development cycle.	User involvement and communication is at a high- degree level between the end user and the engineer.
5	As the model is kind of one-time execution, the reusability factor is least possible here	During execution of the multiple iterations, the possibility of reusability is quite necessary and developed by nature

Benefits

- Spiral development is faster than the traditional model, and so saves cost and effort.
- As risk evaluation is well defined in proper method, larger projects are created and handled in a strategic way. Also, it adds control during all the phases of software development.
- Spiral model produces the intermediate deliverables at end of each iteration. It gives a great opportunity for the end customers to share their feedback during the early stage.

Challenges

- The Spiral model requires expert associates to execute. Apparently, it increases the overall cost, and so, it is not a good fit for smaller projects.
- In the worst cases, Spiral may go on infinitely, which creates wastage on its investment in terms of effort and cost.
- As Spiral has a high degree of complexity, and is relatively difficult to follow strictly, documentation is more due to the execution of multiple intermediate phases of the development cycle.

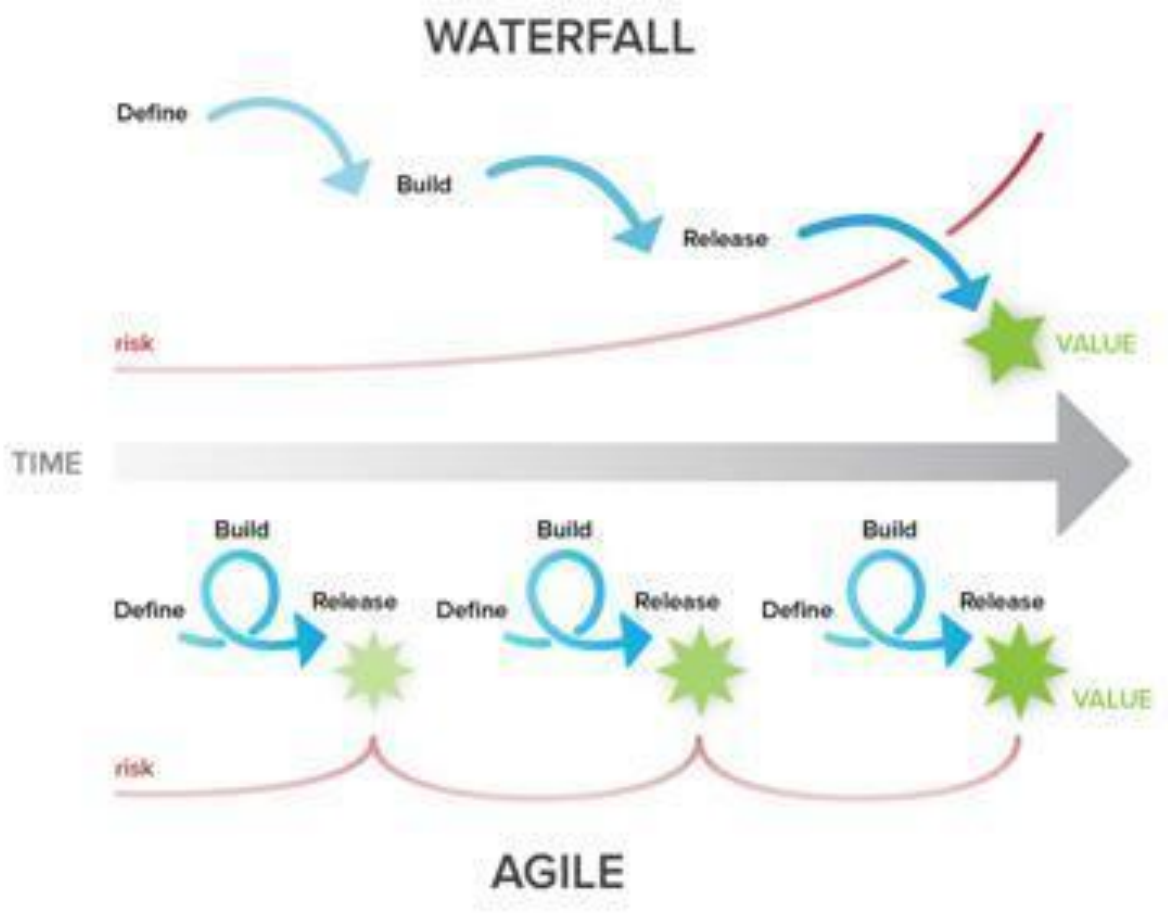
Usage recommendation

- It is highly recommended to use it in large- sized projects where the software needs continuous risk evaluation.
- When requirements are a bit complicated and require the continuous clarification, along with the software changes being quite dynamic, the Spiral model is the best fit.
- In the same line, when the software development life cycle has sufficient time to get end user feedback for frequent releases, the Spiral model has the highest advantage to roll out.

Agile model

- In the legacy Waterfall model, software is developed in the sequential model.
- On completion of the requirements set, the process moves into design, followed by coding. When we enter the design phase, any change to the requirements is not allowed.
- As the name suggests, primarily, the water only flows down, you can't make it go up. This is depicted in the following diagram:

Agile model


WATERFALL
Define
Build
Release
risk
VALUE
TIME
Build
Define
Release
Build
Define
Release
Build
Define
Release
VALUE
risk
AGILE

Agile model

- Scrum is the popular term used in the Agile model.
- Scrum is one of the software development models in the IT industry to support the emerging industry.
- In alignment with the faster and better deliverables in shorter cycle to support the cost efficiency, Scrum is designed with a few small teams of intensive, inter-dependent, self-sustaining characteristics.
- In fact, the deliverables are divided into a small intermediate level, named as Sprint.

Top five reasons to adopt Agile

- Ambiguous requirements
  - In the traditional model, we assume that the customer will identify all requirements during the initial phase. Though it looks reasonable, the customer is not comfortable to sign off the process officially.
  - As an outcome, the customer gives a list of known and probable requirements during the initial phase of software development. It is just to avoid missing them in sign-off mode.

Top five reasons to adopt Agile

- Requirement changes
  - Changes are inevitable.
  - The customers don't know what they want in the beginning, and they eventually come back in the later phases. We assume that the cost of change increases during development, and so, a requirements freeze is absolutely required.
  - In this emerging highly competitive market, requirements will evolve over a period of time.

Top five reasons to adopt Agile

- Big planning is not practical
  - We assume that software is so simple that its development can be easily planned from the beginning to the end phase.
  - In fact, planning should also evolve along with the requirements.
  - The Standish Group Chaos Report 2015 showed that 39% of Agile success versus 11% of Waterfall against all-sized projects.

Top five reasons to adopt Agile

- Software review is better than document
  - Customers are happier to review a working software than a bunch of written documents.

Top five reasons to adopt Agile

- Iterative incremental delivery is preferred
  - Customers can't wait until the completion of the entire project to get the final product.
  - The traditional model assumes to deliver the big final product without the intermediate state. But the customer almost always tends to get a deviation of what they required. It is then too late to rectify it.
  - To resolve this challenge, the Agile model provides delivery to the customers frequently, in short iterations.

Scaled Agile Framework (SAFe)

- Scaled Agile Framework (SAFe) is quite popular, because it is a freely revealed knowledge base of integrated patterns for enterprise-scale Lean-Agile development.
- It has the bright characteristics of a scalable and modular approach.
- It allows an enterprise to leverage this model that suits their needs.
- The SAFe framework synchronizes alignment, collaboration, and delivery for large numbers of agile teams in an enterprise.

Scaled Agile Framework (SAFe)

- SAFe has six core values in its design, which are as follows:
  - Code quality
  - Program execution
  - Business alignment
  - Transparency level
  - Working model

Scaled Agile Framework (SAFe)

- The fundamental theme is built around collaboration and adoption of face-to-face planning and retrospectives of the business.
- As part of the delivery, cross-functional agile teams deliver the working system increments every two weeks.
- There are two different types of SAFe:
  - 3-Level SAFe is for smaller implementations with 100 people or less, or for multiple such programs that do not require significant collaboration.
  - 4-Level SAFe is for solutions that typically require many hundreds of practitioners to develop, deploy, and maintain.

Microsoft open-source strategy to life cycle

- **Open sourcing** and **crowd sourcing** are the latest trends in the software development industry. Though both terms look similar, there is a key difference between open source and crowd source.

Microsoft open-source strategy to life cycle

- Open source is built based on four key pillars
  - Free to use
  - Study
  - Modify
  - Distribute the software for any purpose.
- It creates multiple custom software versions.
- Example: Android mobile platform

Microsoft open-source strategy to life cycle

- Crowd sourcing is built based on the marginal interest as well as free time of a large group of people to build an application.
- This culture benefits the initiator typically without any significant compensation to the participants.
- Example: OpenSignal

Microsoft open-source strategy to life cycle

- Closed source is a complete contradiction to the open source and crowd source culture.
- It is sometimes referred to as the proprietary model.
- By design, closed source methodology is owned by an organization with in-house development to generate revenue for their business.

Closed Source vs Open Source

Microsoft open-source strategy to life cycle

- Microsoft, Apple, and others are the major players among closed source companies
- Linux, Android, and others are prominent among open-source companies.
- Although Microsoft held an identity as a closed source company, it has switched over to the open-source list in recent years.

Driving factors of the open-source model

- Three decades of the Windows platform clearly indicates a closed source company.
- Microsoft initiated open sourcing more of its own technologies such as parts of the .NET platform.
- Now Microsoft is the top organization with the most open-source contributors on GitHub. In fact, Microsoft beats Facebook, Docker, Google, Apache, and many other competitors.

Twin tracks of .NET Framework and .NET Core

- .NET Framework evolved during the early 2000's as a competitive product against the Java platform.
- In a nutshell, the .NET framework is marching forward with a closed source strategy, whereas the .NET Core strategy is towards open source.

Comparing .NET with .NET Core

Sr.No.	.NET	.NET Core
1	Complete development environment for Microsoft .NET Application	As .NET is super set, .NET core will be a subset of the functionalities.
2	Closed source strategy.	Open-source strategy.
3	Runs on Windows platform only.	True cross-platform; supports all major OSs like Windows, Linux, Mac, and the like.
4	Proprietary compiler.	Open-source compiler, Roslyn.
5	Distributed with Windows.	Distributed with Application.
6	Hosting server IIS, runs only on Windows platform.	Open-source hosting server Kestrel, runs across multiple platforms beyond Windows.

Current stack of open-source

- Azure is the best use case to describe open-source tools and techniques by Microsoft
- Azure clearly has become a multiplatform cloud. Indeed, 25 percent of Azure virtual machines (VM) are now running Linux. As a top note, Azure now supports five Linux servers as VMs: CoreOS, CentOS, Oracle Linux, SUSE, and Ubuntu.
- Interestingly, Microsoft is in the process of supporting Docker and Kubernetes container management on Windows and Azure platforms.

Some of the interesting tools and techniques

- .NET Core,
- TypeScript
- R Tools for Visual Studio
- Azure's Service Fabric
- Visual Studio Code IDE,
- Team Explorer Everywhere for Eclipse
- Computational Network Toolkit for deep learning, AIX tools you can use to build AI in Minecraft
- and many more.

Summary

- You can understand the various types of software development lifecycle processes used in the industry. They includes Waterfall, Spiral, Agile and Scaled Agile framework.
- You learnt the insight of each model along with the comparison of few instances.
- Next chapter, we will discuss the enterprise practices in software development like maintainability, reusability, testability, and performance optimization in an enterprise application.

Lab 02

Reference

- Senthilvel, G., Khan, O. M. A., & Qureshi, H. A. (2017). Enterprise Application Architecture with. NET Core. Packt Publishing Ltd.

Q&A

This Photo by Unknown Author is licensed under CC BY-NC-ND